

Нейросетевые методы искусственного интеллекта

Нейронные сети

Обучение нейронных сетей: прямое и обратное распространение ошибки

Лекция 2



2 Обучение нейронных сетей: прямое и обратное распространение ошибки

В предыдущей лекции мы познакомились с архитектурой нейронных сетей, перцептроном и его возможностями. Мы убедились, что даже один перцептрон может выполнять простые логические операции, а их комбинация в многослойную сеть позволяет моделировать функции любой сложности. Теперь мы подходим к ключевому вопросу: *как обучать такие многослойные сети?* Ответ на этот вопрос лежит в основе обучения нейронных сетей. Мы имеем набор обучающих данных — пар «вход — правильный выход». Процесс обучения заключается в настройке параметров сети так, чтобы её выходы максимально приближались к правильным ответам из обучающей выборки.

2.1 Дифференцируемые ступенчатые функции

Вспомним, что перцептрон использует ступенчатую функцию Хевисайда в качестве функции активации. Она имеет серьёзный недостаток — разрыв в точке $x = 0$, что делает её недифференцируемой. Почему это важно? Как мы увидим далее, обучение нейронной сети требует вычисления градиента функции потерь относительно весов. Градиент — это вектор частных производных, следовательно, для его вычисления необходима дифференцируемость.

Нам нужны функции, которые, с одной стороны, имитируют ступенчатую функцию (позволяют принимать бинарные решения), а с другой — являются гладкими и дифференцируемыми. В этом разделе мы обсудим несколько функций, которые являются дифференцируемыми и при этом могут имитировать ступенчатую функцию Хевисайда. Наиболее значимая среди них — *сигмоидная функция*.

2.1.1 Сигмоидная функция

Сигмоидная функция получила такое название из-за её характерной S-образной формы. Это одна из наиболее часто используемых функций активации в скрытых слоях нейронных сетей, особенно на заре развития глубокого обучения. Соответствующее уравнение имеет вид:

$$\sigma(x) = \frac{1}{1 + e^{-x}}. \quad (2.1)$$

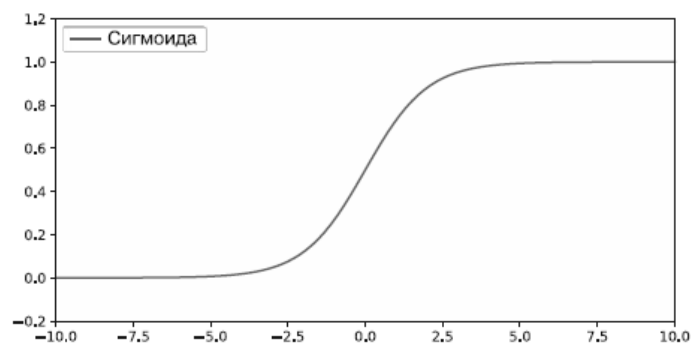


Рисунок 1 — График одномерной сигмоидной функции

Как видно из рисунка 1, сигмоида гладко отображает любое действительное число в интервал $(0, 1)$.

Сигмоидная функция обладает несколькими свойствами:

- *Выражение через положительную экспоненту:*

$$\sigma(x) = \frac{e^x}{1 + e^x}. \quad (2.2)$$

Это выражение легко получить, умножив числитель и знаменатель уравнения (2.1) на e^x .

- *Симметрия относительно центра:*

$$\sigma(-x) = \frac{e^{-x}}{1 + e^{-x}} = \frac{1 + e^{-x} - 1}{1 + e^{-x}} = 1 - \frac{1}{1 + e^{-x}} = 1 - \sigma(x). \quad (2.3)$$

Это свойство показывает, что функция симметрична относительно точки $(0, 0.5)$.

- *Производная сигмоиды:*

$$\begin{aligned} \sigma'(x) &= \left(\frac{1}{1 + e^{-x}} \right)' = \left(\left(1 + e^{-x} \right)^{-1} \right)' = \frac{-1}{(1 + e^{-x})^2} \cdot \left(1 + e^{-x} \right)' = \\ &= \frac{1}{(1 + e^{-x})^2} \cdot e^{-x} = \frac{1}{1 + e^{-x}} \cdot \frac{e^{-x}}{1 + e^{-x}} = \sigma(x)(1 - \sigma(x)). \end{aligned} \quad (2.4)$$

Производная сигмоиды выражается через саму функцию, что значительно упрощает вычисления при градиентном спуске.

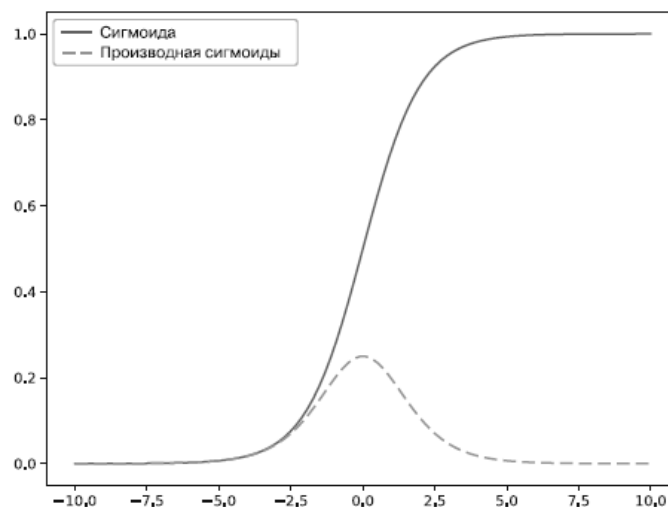


Рисунок 2 — График одномерной сигмоидной функции и её производной

На рисунке 2 показан график производной сигмоиды, наложенный на график самой функции. Производная имеет максимальное значение (0.25) в середине сигмоиды, где функция поднимается более или менее линейно, и стремится к нулю на концах, где функция насыщается и почти не меняется.

Уравнение (2.1) можно параметризовать как:

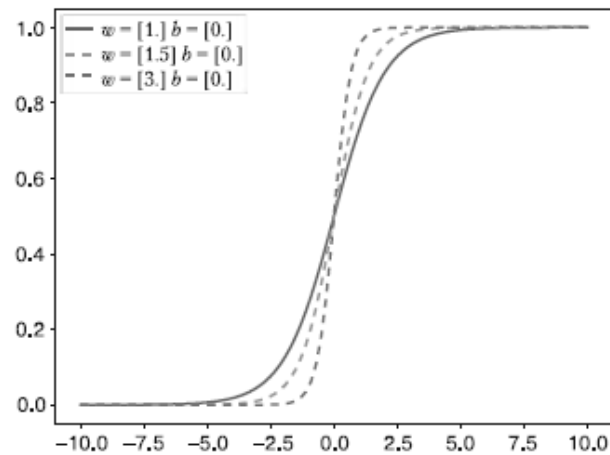
$$\sigma(x) = \frac{1}{1 + e^{-(wx+b)}}. \quad (2.5)$$

Здесь:

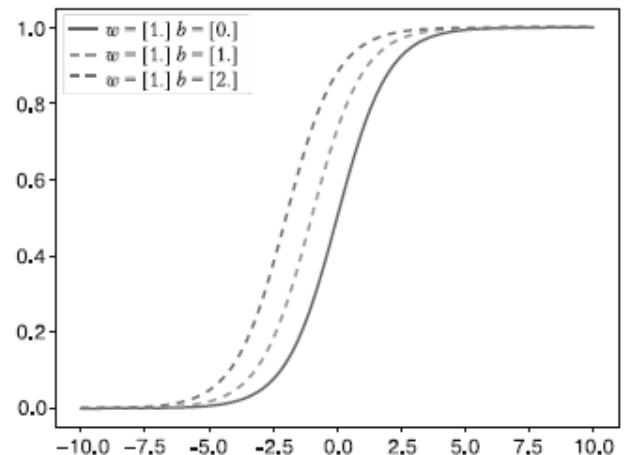
- Параметр w управляет крутизной наклона S-образной кривой. При больших w сигмоида становится почти неотличимой от ступенчатой функции, но остаётся дифференцируемой.

- Параметр b управляет положением кривой (сдвигает её вдоль оси x).

На рисунке 3 показано, как выглядит параметризованная сигмоидная кривая при различных значениях параметров.



а — Выход одномерного перцептрона — различные веса



б — Выход одномерного перцептрона — различные смещения

Рисунок 3 — Сигмоидные кривые, соответствующие различным значениям параметров

2.1.2 Функция гиперболического тангенса

Альтернативой сигмоиде является функция гиперболического тангенса ($\tanh$). Она очень похожа на сигмоиду, но имеет диапазон выходных значений $[-1, 1]$. Уравнение функции $\tanh$ задается как:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}. \quad (2.6)$$

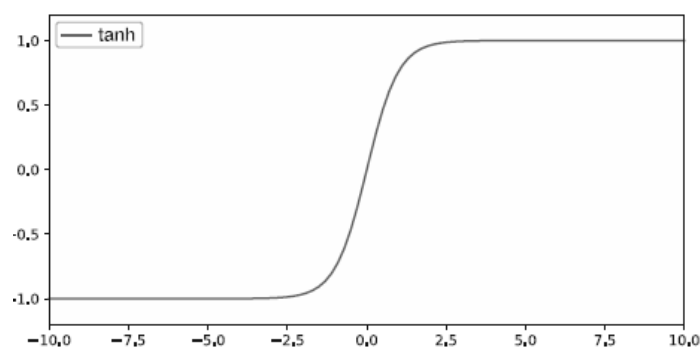


Рисунок 4 — График одномерной функции $\tanh$

Функция $\tanh$ обладает свойствами:

- *Связь с сигмойдой:* гиперболический тангенс может быть выражен через сигмоиду:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{(e^x - e^{-x}) : e^x}{(e^x + e^{-x}) : e^x} = \frac{1 - e^{-2x}}{1 + e^{-2x}} = \frac{2}{1 + e^{-2x}} - 1 = 2\sigma(2x) - 1. \quad (2.7)$$

- *Нечетность и симметрия:* так как $\tanh(x)$ — нечетная функция, то

$$\tanh(-x) = \frac{e^{-x} - e^x}{e^{-x} + e^x} = -\frac{e^x - e^{-x}}{e^x + e^{-x}} = -\tanh(x) \quad (2.8)$$

что обеспечивает симметрию относительно начала координат.

- *Производная гиперболического тангенса:*

$$\begin{aligned}\tanh'(x) &= \frac{(e^x + e^{-x})^2 - (e^x - e^{-x})^2}{(e^x + e^{-x})^2} = \\ &= 1 - \left(\frac{e^x - e^{-x}}{e^x + e^{-x}} \right)^2 = 1 - \tanh^2(x).\end{aligned}\tag{2.9}$$

Как и в случае сигмоиды, производная выражается через саму функцию, что удобно для обратного распространения ошибки.

Преимущество $\tanh$ перед сигмоидой становится очевидным при сравнении их производных. Производная функции $\tanh$ вблизи $x = 0$ значительно выше, чем у сигмоиды. Это обеспечивает более быструю сходимость процесса обучения, так как обновления весов происходят большими шагами. Это особенно эффективно, когда данные центрированы вокруг нуля (предварительная стандартизация).

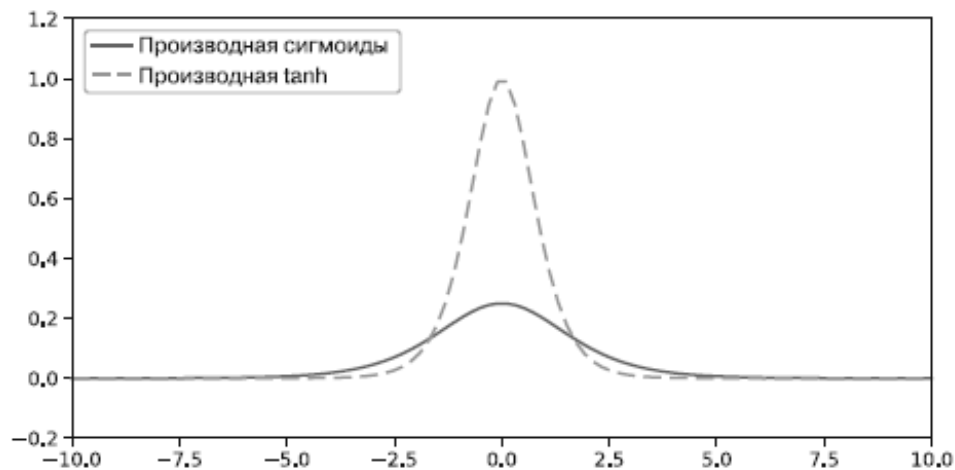


Рисунок 5 — График производных одномерных функций: сигмоидной и гиперболического тангенса

2.2 Линейные слои

В архитектурах нейронных сетей применяются слои разных типов: сверточные, рекуррентные, нормализующие. Однако базовым элементом многих архитектур является *линейный* (или *полносвязный*) слой. В таком слое каждый нейрон последующего слоя соединен с каждым нейроном предыдущего. Если предыдущий слой содержит m нейронов, а следующий — n , то число связей равно mn , и каждой связи соответствует отдельный весовой коэффициент.

На рисунке 6 показан линейный слой, который является срезом большего многослойного перцептрона. Полная архитектура многослойного перцептрона показана на рисунке 7. В обозначениях весов верхний индекс указывает номер слоя, нижний индекс — пару «приемник–источник».

Вес соединения, связывающего k -й нейрон в слое $(l-1)$ с j -м нейроном в слое l , обозначается $w_{jk}^{(l)}$. Здесь нижний индекс j означает приемник, а k — источник.

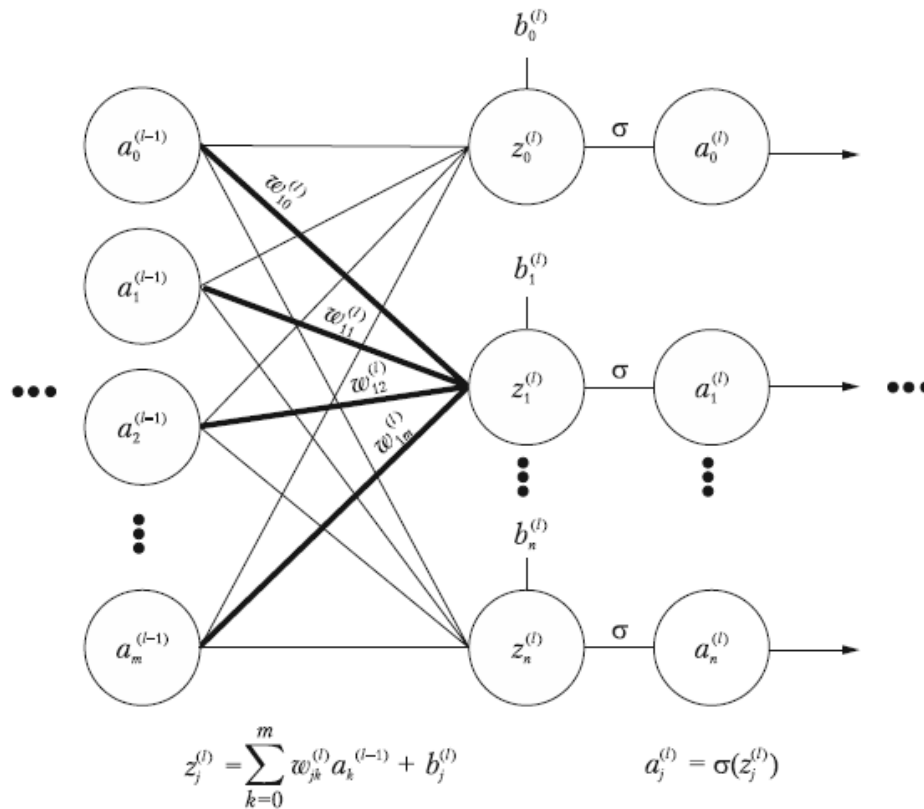


Рисунок 6 — Линейный слой l , получающий выход слоя $(l - 1)$. Веса, принадлежащие строке j матрицы весов (поступающие от всех входных нейронов в слое $(l - 1)$, которые суммируются для формирования выхода нейрона j), выделены жирным

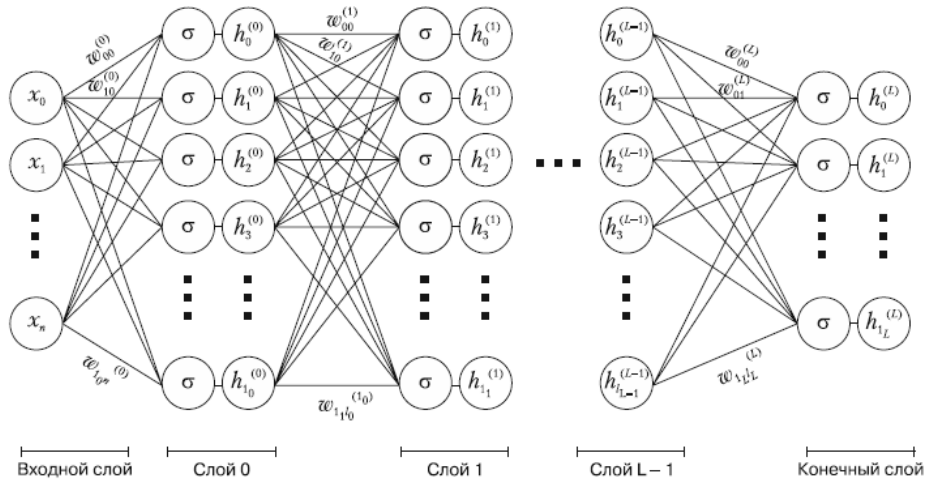


Рисунок 7 — Многослойная нейронная сеть

2.2.1 Выражение линейных слоев как умножение матрицы на вектор

Рассмотрим перцептрон в составе многослойной сети. Как было показано ранее, один перцептрон берет взвешенную сумму своих входов и применяет к ней ступенчатую функцию. В многослойном перцептроне входы любого перцептрона в l -м слое поступают из предыдущего, $(l - 1)$ -го слоя.

Пусть $a_0^{(l-1)}, a_1^{(l-1)}, \dots, a_m^{(l-1)}$ обозначают выходы m нейронов в слое $(l - 1)$ (столбец узлов слева на рисунке 6). И пусть $a_0^{(l)}, a_1^{(l)}, \dots, a_n^{(l)}$ обозначают выходы n нейронов в слое l . Обратите

внимание на то, что для обозначения выходов нейронов мы обычно используем символ a , который означает activation (активация). Теперь рассмотрим j -й нейрон в слое l . Например, взгляните на рисунок 6 — обратите внимание на веса, входящие в него, и активации, откуда эти веса исходят. Его выход — это $a_j^{(l)}$, где:

$$\left. \begin{aligned} z_j^{(l)} &= \sum_{k=0}^m w_{jk}^{(l)} a_k^{(l-1)} + b_j^{(l)}, \\ a_j^{(l)} &= \sigma(z_j^{(l)}) \end{aligned} \right\} \quad \text{при } j = 0 \dots n.$$

Мы можем заменить сумму в этих уравнениях скалярным произведением векторов с весами и активациями:

$$\left. \begin{aligned} z_j^{(l)} &= \begin{bmatrix} w_{j0}^{(l)} & w_{j1}^{(l)} & \dots & w_{jm}^{(l)} \end{bmatrix} \begin{bmatrix} a_0^{(l-1)} \\ a_1^{(l-1)} \\ \vdots \\ a_m^{(l-1)} \end{bmatrix} + b_j^{(l)}, \\ a_j^{(l)} &= \sigma(z_j^{(l)}) \end{aligned} \right\} \quad \text{при } j = 0 \dots n.$$

Совокупность уравнений для всех j компактно представляется в матрично-векторной форме:

$$\vec{z}^{(l)} = W^{(l)} \vec{a}^{(l-1)} + \vec{b}^{(l)}, \quad \vec{a}^{(l)} = \sigma(\vec{z}^{(l)}), \quad (2.10)$$

где:

- $W^{(l)}$ — матрица $n \times m$, представляющая веса всех соединений, связывающих слой $(l-1)$ со слоем l :

$$W^{(l)} = \begin{bmatrix} w_{00}^{(l)} & w_{01}^{(l)} & \dots & w_{0m}^{(l)} \\ w_{10}^{(l)} & w_{11}^{(l)} & \dots & w_{1m}^{(l)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{n0}^{(l)} & w_{n1}^{(l)} & \dots & w_{nm}^{(l)} \end{bmatrix}, \quad (2.11)$$

- $\vec{a}^{(l)}$ — активации для всего слоя l . Применение сигмоидной функции к вектору эквивалентно применению ее по отдельности к каждому элементу вектора:

$$\vec{a}^{(l)} = \begin{bmatrix} a_0^{(l)} \\ a_1^{(l)} \\ \vdots \\ a_n^{(l)} \end{bmatrix}; \quad \vec{a}^{(l-1)} = \begin{bmatrix} a_0^{(l-1)} \\ a_1^{(l-1)} \\ \vdots \\ a_m^{(l-1)} \end{bmatrix}; \quad \vec{z}^{(l)} = \begin{bmatrix} z_0^{(l)} \\ z_1^{(l)} \\ \vdots \\ z_n^{(l)} \end{bmatrix}; \quad \sigma(\vec{z}^{(l)}) = \begin{bmatrix} \sigma(z_0^{(l)}) \\ \sigma(z_1^{(l)}) \\ \vdots \\ \sigma(z_n^{(l)}) \end{bmatrix}.$$

Матрично-векторная форма записи избавляет от необходимости иметь дело с индексами во время работы со всеми весами, смещениями, активациями.

2.2.2 Прямое распространение для многослойного перцептрона с линейными слоями

Уравнение (2.10) описывает *прямого распространения* (forward propagation) в одном линейном слое. Конечный выход многослойного перцептрона с полносвязанными (то есть линейными) слоями $0 \dots L$ для входа $\vec{x}$ можно получить повторным применением следующего уравнения:

$$\text{MLP}(\vec{x}) = \vec{a}^{(L)} = \vec{y} = \sigma \left(W^{(L)} \dots \sigma \left(W^{(1)} \sigma \left(W^{(0)} \vec{x} + \vec{b}^{(0)} \right) + \vec{b}^{(1)} \right) \dots + \vec{b}^{(L)} \right). \quad (2.12)$$

На практике это выражение не вычисляется явно в виде единой композиции, а реализуется итерационно, слой за слоем:

$$\begin{aligned} \vec{a}^{(0)} &= \sigma \left(W^{(0)} \vec{x} + \vec{b}^{(0)} \right); \\ \vec{a}^{(1)} &= \sigma \left(W^{(1)} \vec{a}^{(0)} + \vec{b}^{(1)} \right); \\ &\dots \\ \vec{a}^{(L)} &= \sigma \left(W^{(L)} \vec{a}^{(L-1)} + \vec{b}^{(L)} \right). \end{aligned} \quad (2.13)$$

Легко заметить, что уравнение (2.13) является уравнением (2.10), выраженным иначе.

Тщательное изучение этих уравнений выявляет свойство. Сложное уравнение (2.12) *никогда не оценивается явно*. Вместо этого оценка выходов производится послойно, согласно уравнению (2.13). Каждый слой можно оценить, взяв выходные данные предыдущего слоя и используя их в качестве входных данных текущего слоя. Никакие другие входные данные не нужны. То есть мы можем вычислить $\vec{a}^{(0)}$ непосредственно из входных данных $\vec{x}$, затем $\vec{a}^{(1)}$ из $\vec{a}^{(0)}$, $\vec{a}^{(2)}$ из $\vec{a}^{(1)}$ и т. д., вплоть до $\vec{a}^{(L)}$, который является конечным выходом многослойного перцептрона. Во время оценки в любой момент нужно хранить в памяти только предыдущий и текущий слой. Этот процесс значительно упрощает реализацию и концептуализацию и известен как *прямое распространение ошибки*.

2.3 Обучение как оптимизация

Обучение нейронной сети — это итеративный процесс, с помощью которого подбираются параметры нейронной сети. Цель — оценить параметры (веса и смещения) так, чтобы для входных обучающих данных нейронная сеть выдавала на выходе данные, максимально близкие к известным истинным выходным данным.

В целом итеративный процесс постепенно улучшает получаемые выходные данные (приближает их к цели). На каждой итерации в значения параметров вносятся небольшие корректировки. Здесь под параметрами подразумеваются веса и смещения в многослойном перцептроне, $w_k^{(l)}$ и $b_j^{(l)}$. Параметры корректируются так, чтобы на каждой итерации выходные данные, получаемые в ответ на ввод входных обучающих данных, немного приближались к истинным выходным данным. После множества итераций параметры, как мы надеемся, сходятся к оптимальным значениям.

Итерационный процесс может не сходиться к наилучшим возможным значениям параметров. Обучение может полностью уйти в сторону или застрять в локальном минимуме. Не существу-

ет хорошего способа узнать, были ли достигнуты оптимальные значения (глобальный минимум) весов и смещений. Для выяснения этого обычно на вход нейронной сети подаются тестовые данные, и если она выдает удовлетворительные результаты, то обучение прекращается. Тестовые данные не должны участвовать в процессе обучения. В случае неудачи, если сеть не достигла желаемого уровня точности, принято увеличивать выборку с обучающими данными, изменять функцию потерь и/или пробовать использовать другую архитектуру. Простое повторное обучение сети с другими случайными начальными весами и смещениями тоже может дать положительный результат. Это экспериментальная наука, в которой движение вперед осуществляется методом проб и ошибок.

2.3.1 Функция потерь и её минимизация

Как узнать, в какую сторону и насколько корректировать значения параметров на каждой итерации? Для этого определяется функция потерь, она же функция ошибки. Существует много популярных представлений функций потерь, и позже мы познакомимся с некоторыми другими, однако все они имеют общее свойство: когда выход нейронной сети приближается к известным истинным выходам, потери уменьшаются и наоборот. Определив функцию потерь, мы получаем четкое количественное определение цели обучения нейронной сети, которая состоит в том, чтобы минимизировать общие потери по всему набору обучающих данных.

Пусть дан набор обучающих данных T , содержащий пары «вход, истинный выход» $T = \{(\vec{x}, \vec{y})\}$. В таком случае функцию потерь можно выразить как:

$$L = \frac{1}{2} \sum_{i=1}^n (\vec{y} - \bar{\vec{y}})^2, \quad (2.14)$$

где:

$$\bar{\vec{y}} = \text{MLP}(\vec{x}),$$

согласно уравнению (2.12).

Рассмотрим уравнение (2.10), мы можем преобразовать в вектор матрицу весов каждого слоя $W^{(l)}$, а затем объединить векторы из последовательных слоев в один гигантский вектор $\vec{w}$ всех весов в многослойном перцептроне:

$$\vec{w} = \begin{bmatrix} \bar{w}_{00}^{(0)} & \bar{w}_{01}^{(0)} & \cdots & \bar{w}_{00}^{(1)} & \bar{w}_{01}^{(1)} & \cdots & \bar{w}_{00}^{(L)} & \bar{w}_{01}^{(L)} & \cdots \end{bmatrix}$$

Аналогично можно сформировать гигантский вектор всех смещений:

$$\vec{b} = \begin{bmatrix} b_0^{(0)} & b_1^{(0)} & \cdots & b_0^{(1)} & b_1^{(1)} & \cdots & b_0^{(L)} & b_1^{(L)} & \cdots \end{bmatrix}.$$

Формально цель обучения заключается в нахождении таких $\vec{w}$ и $\vec{b}$, которые минимизируют функционал L . Классический подход теории оптимизации предполагает решение системы уравнений, получаемых приравнением градиентов к нулю:

$$\nabla_{\vec{w}} L = 0, \quad \nabla_{\vec{b}} L = 0.$$

Однако прямое аналитическое решение этой системы невозможно по следующим причинам:

- Функция потерь L является сложной суперпозицией нелинейных преобразований (в частности,

функций активации), что делает задачу нахождения производных и их приравнивания к нулю крайне громоздкой.

- Даже в упрощённых случаях система уравнений не имеет замкнутого аналитического решения.

Вследствие этого применяется итерационный численный метод — градиентный спуск на поверхности функции потерь, который позволяет последовательно приближаться к точке экстремума.

2.3.2 Геометрия обучения: поверхность потерь и градиентный спуск

Геометрически функцию потерь $L(\vec{w}, \vec{b})$ можно рассматривать как поверхность в многомерном пространстве. Количество измерений этого пространства равно сумме измерений $\vec{w}$ и $\vec{b}$. Пример для двумерного пространства показан на рисунке 8. Лучший способ двигаться к минимуму — двигаться в направлении, где функция убывает быстрее всего. В математике это направление указывает антиградиент (отрицательный градиент). Поэтому на каждой итерации мы корректируем параметры, делая небольшой шаг в сторону, противоположную градиенту. На рисунке 8 путь градиентного спуска показан сплошными стрелками, а произвольный неоптимальный путь — пунктирными.

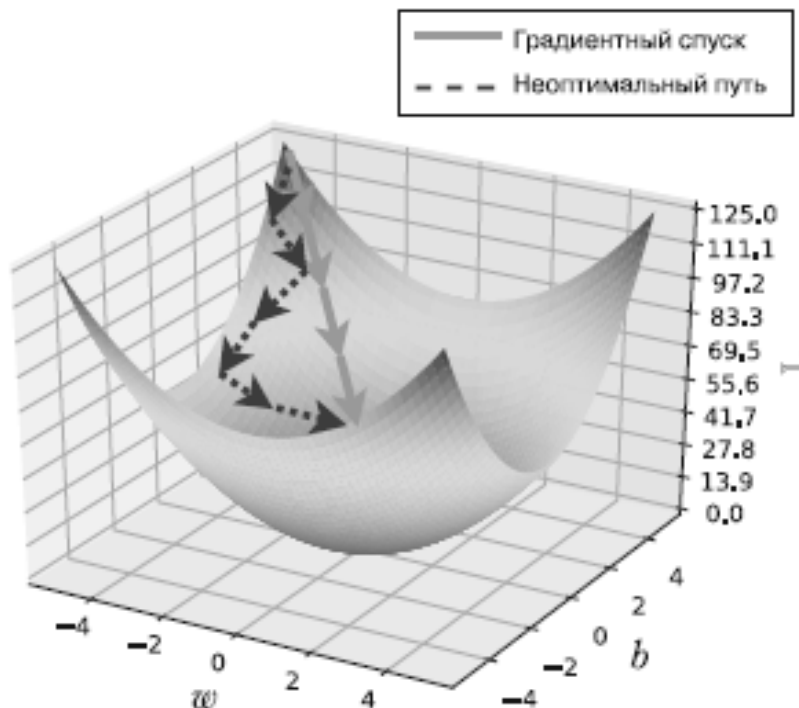


Рисунок 8 — Репрезентативная поверхность функции потерь $L(\vec{w}, \vec{b})$.

Правило обновления для весов и смещений выглядит следующим образом:

$$\vec{w} = \vec{w} - r \nabla_{\vec{w}} L; \quad \vec{b} = \vec{b} - r \nabla_{\vec{b}} L, \quad (2.15)$$

где r — константа. Здесь:

$$\nabla_{\vec{w}} L = \left[\frac{\partial L}{\partial \vec{w}_{jk}^{(l)}} \right] \text{ для всех } l, j, k;$$

$$\nabla_{\vec{b}} L = \left[\frac{\partial L}{\partial \vec{b}_j^{(l)}} \right] \text{ для всех } l, j.$$

Уравнение (2.15) обновления вектора можно выразить через скалярные компоненты:

$$\vec{w}_{jk}^{(l)} = \vec{w}_{jk}^{(l)} - r \frac{\partial L}{\partial \vec{w}_{jk}^{(l)}} \text{ для всех } l, j, k;$$

$$b_j^{(l)} = b_j^{(l)} - r \frac{\partial L}{\partial b_j^{(l)}} \text{ для всех } l, j.$$

Все частные производные необходимо заново вычислять на каждой итерации, поскольку их значения будут меняться.

Чтобы понять, почему движение вдоль отрицательного градиента максимально уменьшает функцию, воспользуемся приближением Тейлора для малого смещения $\delta \vec{w}$:

$$L(\vec{w} - \delta \vec{w}) = L(\vec{w}) - \frac{1}{1!} (\delta \vec{w})^T \nabla_{\vec{w}} L + \frac{1}{2!} (\delta \vec{w})^T H(\delta \vec{w}) + \dots,$$

где матрица H , называемая *матрицей Гессе* или *Гессаном функции*. Поскольку мы остаемся в окрестности $\vec{w}$, расстояние $\|\delta \vec{w}\|$ очень мало. Это означает, что квадратичные и более высокие члены пренебрежимо малы и их можно отбросить (приближение идеально в пределе, когда $\|\delta \vec{w}\| \rightarrow 0$):

$$L(\vec{w} - \delta \vec{w}) \approx L(\vec{w}) - \frac{1}{1!} (\delta \vec{w})^T \nabla_{\vec{w}} L.$$

Из линейной алгебры известно, что скалярное произведение $(\delta \vec{w})^T \nabla_{\vec{w}} L$ минимально (то есть дает наибольшее отрицательное изменение), когда векторы $\delta \vec{w}$ и $\nabla_{\vec{w}} L$ направлены в противоположные стороны. Следовательно, чтобы уменьшить L , мы должны выбрать:

$$\delta \vec{w} = -r \nabla_{\vec{w}} L,$$

что в точности соответствует правилу обновления (2.15). Коэффициент r называется *скоростью обучения*. Чем она выше, тем быстрее происходит спуск и тем выше риск выскочить за пределы области минимума. r — это *настраиваемый гиперпараметр системы*. Аналогичные рассуждения справедливы и для вектора смещений $\vec{b}$. На рисунке 8 сплошные стрелки иллюстрируют этот оптимальный путь, а пунктирные — любой другой, более долгий маршрут.

Поверхность потерь реальных нейронных сетей не является выпуклой и содержит множество локальных минимумов. Градиентный спуск может остановиться в одном из таких локальных минимумов, который может оказаться не глобальным. Эта ситуация показана на рисунке 9.

В отличие от классических методов оптимизации, которые активно пытались избежать локальных минимумов (например, имитация отжига), современный подход к обучению нейросетей более прагматичен. Во-первых, часто локальные минимумы оказываются достаточно «хорошими», чтобы обеспечить приемлемую точность на практике. Во-вторых, если результат не удовлетвори-

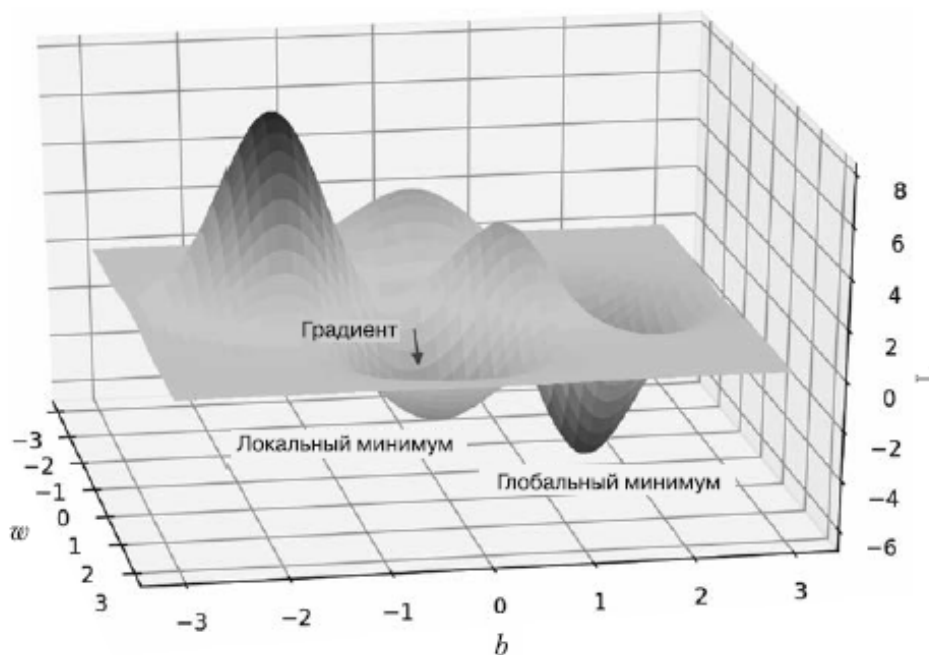


Рисунок 9 — Невыпуклая функция с локальными и глобальными минимумами. В зависимости от выбора начальной точки градиентный спуск может привести в локальный или глобальный минимум

телен, обучение можно просто запустить заново с другой случайной инициализацией параметров, и градиентный спуск с высокой вероятностью приведет к более удачной области поверхности потерь.

2.4 Алгоритм обратного распространения ошибки

Как мы установили, суть градиентного спуска заключается в многократной корректировке весов и смещений с использованием уравнения (2.15). Это эквивалентно многократному обновлению индивидуальных весов и смещений с использованием отдельных частных производных.

Получить аналитическое решение для градиентов $\nabla_{\vec{w}}L$ и $\nabla_{\vec{b}}L$ из уравнения (2.12) и (2.13) — или, что эквивалентно, получить аналитическое решение для частных производных $\frac{\partial L}{\partial w_{jk}^{(l)}}$ и $\frac{\partial L}{\partial b_j^{(l)}}$ — очень сложно. Поэтому мы используем обратное распространение ошибки — алгоритм, позволяющий вычислять градиенты и обновлять веса и смещения послойно по аналогии с прямым распространением.

2.4.1 Обратное распространение в простой сети

Сначала обсудим обратное распространение ошибки на примере простого многослойного перцептрона с одним нейроном на слой. Такая конфигурация позволит избавиться от ненужных нижних индексов в отдельных весах и смещениях, так как между каждой парой последовательных слоев имеются только один вес и одно смещение. Однако верхние индексы, идентифицирующие слои, никогда не исчезнут. Такой многослойный перцептрон показан на рисунке 10. В качестве функции потерь мы используем MSE (2.14), но работаем с одной парой «вход — выход» x_i, y_i . Суммарные потери, складывающиеся из потерь по всем экземплярам обучающих данных, легко получить, повторив те же шаги.

Введем ключевую вспомогательную переменную $\delta^{(l)}$, которая равна частной производной

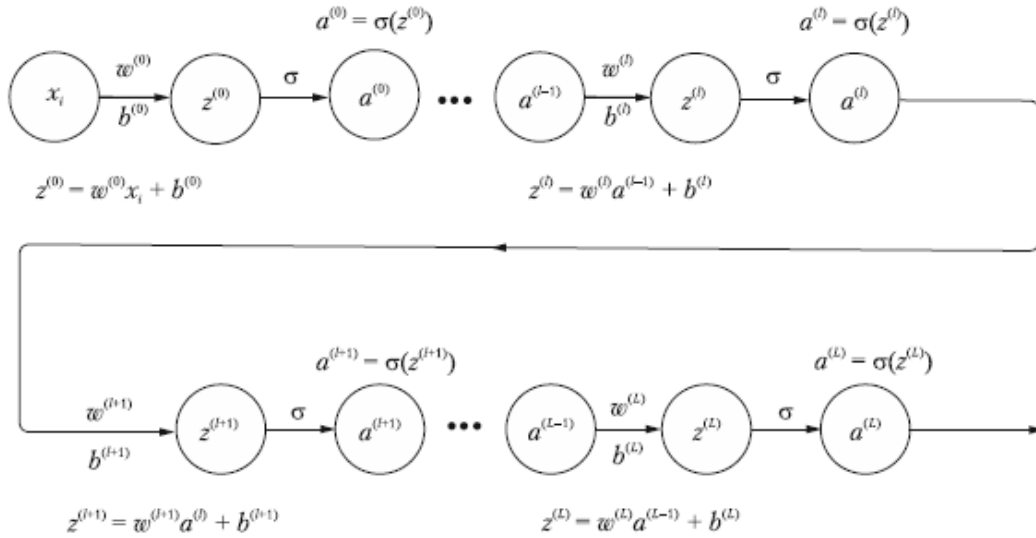


Рисунок 10 — Простой многослойный перцептрон с одним нейроном на слой

функции потерь по предварительно активированному выходу слоя l (т.е. по $z^{(l)}$ до применения функции активации):

$$\delta^{(l)} = \frac{\partial L}{\partial z^{(l)}}, \quad l \in \{0, 1, \dots, L\}. \quad (2.16)$$

Физический смысл $\delta^{(l)}$ — скорость изменения функции потерь при изменении суммарного входа в слой l .

Прямое распространение в такой сети описывается двумя основными уравнениями:

$$z^{(l)} = w^{(l)}a^{(l-1)} + b^{(l)}, \quad (2.17)$$

$$a^{(l)} = \sigma(z^{(l)}). \quad (2.18)$$

Здесь $a^{(l)}$ — выход слоя l после активации, причем $a^{(0)}$ — это входной слой (сами данные x).

В качестве функции потерь для одного обучающего примера (x_i, y_i) будем использовать среднеквадратичную ошибку (MSE):

$$L = \frac{1}{2} (a^{(L)} - \bar{y}_i)^2.$$

Для последнего слоя L вычислим производные по весу и смещению с помощью цепного правила. Начнем с веса. Поскольку L является функцией от $z^{(L)}$, а $z^{(L)}$, в свою очередь, является функцией от $w^{(L)}$, то согласно правилу дифференцирования сложной функции (цепному правилу), производная функции потерь L по весовому коэффициенту $w^{(L)}$ равна:

$$\frac{\partial L}{\partial w^{(L)}} = \frac{\partial L}{\partial z^{(L)}} \cdot \frac{\partial z^{(L)}}{\partial w^{(L)}}. \quad (2.19)$$

Используя определение $\delta^{(L)}$ и уравнение прямого распространения (2.17), получаем:

$$\frac{\partial L}{\partial w^{(L)}} = \delta^{(L)} a^{(L-1)}. \quad (2.20)$$

Аналогично для смещения:

$$\frac{\partial L}{\partial b^{(L)}} = \frac{\partial L}{\partial z^{(L)}} \cdot \frac{\partial z^{(L)}}{\partial b^{(L)}} = \delta^{(L)} \cdot 1 = \delta^{(L)}. \quad (2.21)$$

Теперь вычислим саму вспомогательную переменную для последнего слоя:

$$\delta^{(L)} = \frac{\partial L}{\partial z^{(L)}} = \frac{\partial L}{\partial a^{(L)}} \cdot \frac{\partial a^{(L)}}{\partial z^{(L)}}. \quad (2.22)$$

Производная функции потерь по выходу сети:

$$\frac{\partial L}{\partial a^{(L)}} = a^{(L)} - \bar{y}_i. \quad (2.23)$$

Производная сигмоидальной функции активации:

$$\frac{da(z)}{dz} = \frac{d\sigma(z)}{dz} = \sigma(z) (1 - \sigma(z)) = a^{(L)} (1 - a^{(L)}). \quad (2.24)$$

Таким образом:

$$\delta^{(L)} = (a^{(L)} - \bar{y}_i) \cdot a^{(L)} (1 - a^{(L)}). \quad (2.25)$$

Для произвольного слоя l градиенты по весу и смещению имеют аналогичный вид:

$$\frac{\partial L}{\partial w^{(l)}} = \delta^{(l)} a^{(l-1)}, \quad (2.26)$$

$$\frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}. \quad (2.27)$$

Однако теперь $\delta^{(l)}$ необходимо выразить через $\delta^{(l+1)}$ следующего слоя. Используем цепное правило:

$$\delta^{(l)} = \frac{\partial L}{\partial z^{(l)}} = \frac{\partial L}{\partial z^{(l+1)}} \cdot \frac{\partial z^{(l+1)}}{\partial a^{(l)}} \cdot \frac{\partial a^{(l)}}{\partial z^{(l)}}. \quad (2.28)$$

Здесь:

- $\frac{\partial L}{\partial z^{(l+1)}} = \delta^{(l+1)}$;
- $\frac{\partial z^{(l+1)}}{\partial a^{(l)}} = w^{(l+1)}$ (из уравнения (2.17));
- $\frac{\partial a^{(l)}}{\partial z^{(l)}} = a^{(l)} (1 - a^{(l)})$ (производная сигмоиды).

Подставляя, получаем рекуррентную формулу для распространения δ в обратном направлении:

$$\delta^{(l)} = \delta^{(l+1)} w^{(l+1)} a^{(l)} (1 - a^{(l)}). \quad (2.29)$$

Теперь мы имеем все необходимые компоненты для организации эффективного алгоритма:

1. *Прямой проход*: последовательно вычисляем $z^{(l)}$ и $a^{(l)}$ для всех слоев $l = 0, 1, \dots, L$ с использованием уравнений (2.17) и (2.18). Сохраняем все значения $a^{(l)}$.
2. *Обратный проход*:

2.1. Вычисляем $\delta^{(L)}$ по формуле (2.25).

- 2.2. Для $l = L - 1, L - 2, \dots, 0$ вычисляем $\delta^{(l)}$ по формуле (2.29).
- 2.3. Для каждого слоя l вычисляем градиенты по формулам (2.26) и (2.27).
- 2.4. Обновляем веса и смещения с использованием уравнения (2.15).

Этот процесс, движущийся от выхода сети к входу, называется *обратным распространением ошибки*. На каждом шаге мы можем сразу обновлять параметры слоя, так как дальнейшие вычисления не требуют старых значений градиентов.

2.4.2 Обратное распространение в общей сети

Обобщим полученные результаты на случай сети с произвольным числом нейронов в каждом слое.

Прямое распространение описывается уравнениями:

$$z_j^{(l)} = \sum_{k=0}^m w_{jk}^{(l)} a_k^{(l-1)} + b_j^{(l)}, \quad a_j^{(l)} = \sigma(z_j^{(l)}), \quad (2.30)$$

или в векторно-матричной форме:

$$\vec{z}^{(l)} = W^{(l)} \vec{a}^{(l-1)} + \vec{b}^{(l)}, \quad \vec{a}^{(l)} = \sigma(\vec{z}^{(l)}). \quad (2.31)$$

Функция потерь для одного обучающего примера теперь имеет вид:

$$L = \frac{1}{2} \|\vec{a}^{(L)} - \vec{y}\|^2 = \frac{1}{2} \sum_j (a_j^{(L)} - \bar{y}_j)^2, \quad (2.32)$$

где суммирование ведется по всем нейронам выходного слоя.

Вспомогательные переменные теперь определяются для каждого нейрона:

$$\delta_j^{(l)} = \frac{\partial L}{\partial z_j^{(l)}}, \quad \forall j \in \{0, \dots, n_l\}. \quad (2.33)$$

Для последнего слоя, используя цепное правило, получаем:

$$\delta_j^{(L)} = \frac{\partial L}{\partial z_j^{(L)}} = \frac{\partial L}{\partial a_j^{(L)}} \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} = (a_j^{(L)} - \bar{y}_j) a_j^{(L)} (1 - a_j^{(L)}), \quad (2.34)$$

или в векторной форме:

$$\vec{\delta}^{(L)} = (\vec{a}^{(L)} - \vec{y}) \circ \vec{a}^{(L)} \circ (1 - \vec{a}^{(L)}), \quad (2.35)$$

где $\circ$ обозначает произведение Адамара (поэлементное умножение векторов).

Для произвольного слоя l вспомогательная переменная вычисляется через следующий слой. Используя цепное правило и учитывая, что $z_j^{(l)}$ влияет на все нейроны следующего слоя, получаем:

$$\delta_j^{(l)} = \frac{\partial L(z_0^{(l+1)}, z_1^{(l+1)}, z_2^{(l+1)}, \dots)}{\partial z_j^{(l)}} = \sum_k \frac{\partial L}{\partial z_k^{(l+1)}} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} = \sum_k \frac{\partial L}{\partial z_k^{(l+1)}} \frac{\partial z_k^{(l+1)}}{\partial a_j^{(l)}} \frac{\partial a_j^{(l)}}{\partial z_j^{(l)}}. \quad (2.36)$$

Подставляя $\frac{\partial L}{\partial z_k^{(l+1)}} = \delta_k^{(l+1)}$, $\frac{\partial z_k^{(l+1)}}{\partial a_j^{(l)}} = w_{kj}^{(l+1)}$ и $\frac{\partial a_j^{(l)}}{\partial z_j^{(l)}} = a_j^{(l)}(1 - a_j^{(l)})$, получаем:

$$\delta_j^{(l)} = \sum_k \delta_k^{(l+1)} w_{kj}^{(l+1)} a_j^{(l)} (1 - a_j^{(l)}), \quad (2.37)$$

или в векторно-матричной форме:

$$\vec{\delta}^{(l)} = \left(W^{(l+1)}\right)^T \vec{\delta}^{(l+1)} \circ \vec{a}^{(l)} \circ (1 - \vec{a}^{(l)}). \quad (2.38)$$

Градиенты функции потерь по весам и смещениям выражаются через вспомогательные переменные:

$$\frac{\partial L}{\partial w_{jk}^{(l)}} = \delta_j^{(l)} a_k^{(l-1)}, \quad (2.39)$$

$$\nabla_{W^{(l)}} L = \vec{\delta}^{(l)} \left(\vec{a}^{(l-1)}\right)^T, \quad (2.40)$$

$$\frac{\partial L}{\partial b_j^{(l)}} = \delta_j^{(l)}, \quad (2.41)$$

$$\nabla_{\vec{b}^{(l)}} L = \vec{\delta}^{(l)}. \quad (2.42)$$

Эти формулы обобщают алгоритм обратного распространения на случай сетей произвольной архитектуры. Алгоритм остается тем же: прямой проход для вычисления всех активаций, затем обратный проход для вычисления вспомогательных переменных и градиентов, начиная с последнего слоя и двигаясь к первому.